CS59300CVD Assignment 2

Due date: Friday, Feb. 27, 11:59 PM

This assignment was adapted from Svetlana Lazebnik's course on computer vision.

Homework Policy: Each student should write up their solutions independently, no copying of any form is allowed. If you have used online resources to help with answering the questions, but have come up with your own answers please properly reference the resources. You are allowed to use online resources as long as the answer is your own and properly acknowledge it. Please refer to the course website for policy on academic honesty. Finally, You MUST USE the provided LaTeX template to typeset the report.

1 Assignment Details

1.1 Part 1: Learnable Alignment

Objective: This programming section follows the settings of Assignment 1. However, instead of searching for the best displacements $(\delta x, \delta y)$ via brute force, we will formalize this problem using a spatial transformer and solve it via gradient descent.

Method description: In the following description, we discuss only aligning the image of one channel to another. The implementation of aligning the other channel should be the same.

Given an input image (a single channel) $\mathbf{A}$ and a target image $\mathbf{B}$, the objective function of Assignment 1 is to find

$$\delta x^*, \delta y^* = \arg \min_{\delta x \in \mathcal{X}, \delta Y \in \mathcal{Y}} L(\text{shift}(\mathbf{A}, \delta x, \delta y), \mathbf{B}), \tag{1}$$

where L is the evaluation metrics, shift$(\cdot)$ is the shifting function, and $\mathcal{X}, \mathcal{Y}$ are the search space.

Using the idea of spatial transformer, we can replace shift$(\cdot)$ with a differentiable operation approximation. In other words, we can attempt to solve the same problem using gradient descent! **Hint**: `torch.nn.functional.affine_grid` and `torch.nn.functional.grid_sample` to be very helpful. Please refer to the torch documentation for more details.

Your algorithm should support the following metrics as loss functions.

1. Mean square error (MSE).
2. Normalized cross-correlation (NCC).

Please test your alignment algorithm on the provided images in `data/part1`. The starter code for this part is the same as assignment 1.

1.2 Part 2: Scale-space blob detection

Objective: This programming section is designed to practice what we learned in Lecture 7 to Lecture 13. We will implement a Laplacian blob detector as shown in Fig. 1.

Method description: Please refer to Lecture 13 for the detailed introduction of Laplacian blob detector. In short, the computation of Laplacian blob detector requires the following steps:

Figure 1: Visualization of input and output on the image ‘butterfly.jpg‘.

1. Convert the input image to grayscale.
2. Generate a Laplacian of Gaussian filter.
3. Build a Laplacian scale space, starting with some initial scale and going for n iterations: Filter image with scale-normalized Laplacian at current scale. Save square of Laplacian response for current level of scale space. Increase scale by a factor k. **Note:** For an efficient implementation, instead of increasing the kernel size by a factor of k, you should downsample the image by a factor $\frac{1}{k}$. In that case, you will have to upsample the result or do some interpolation in order to find maxima in scale space. **Pro tip:** It is encouraged that you write an unit-test to validate that downsampling and increasing the kernel size leads to the same result.
4. Perform non-maximum suppression in scale space.
5. Visualize the locations and the scales of the keypoints.

In the starter code, we provide a drawing function `draw_allcircles`. Please use this function when visualizing the results and test your blob detection algorithm on the provided images in `data/part2`.

Pro tip: Feel free to add your own helper functions to keep the code organized and readable!

2 What to include in the report? (10 points)

2.1 Part 1

1. Mathematically describe how you solve the optimization problem in Eq. (1) using gradient descent. Clearly define all introduced notations. (1 point)
2. What implementation choices, e.g. learning rate, or the number of update steps, did you make? How did they affect the quality of the result and the speed of computation? What are some artifacts and/or limitations of your implementation, and what are possible reasons for them? (2 points)
3. For each image, you must include in your report the colorized output and the learned displacement vectors used to align the channels. (2 points)

2.2 Part 2

1. What implementation choices, e.g., kernel size, sigma, or the number of pyramid layers, did you make? How did they affect the quality of the result and the speed of computation? What are some artifacts and/or limitations of your implementation, and what are possible reasons for them? (3 points)

2. For each provided image, you must include the colorized output with all the visualized key-points using the providing function `draw_allcircles`. (2 points)

While code is not explicitly graded. Poorly commented or written code will result in points deducted from the previous three parts. For example, if the grader cannot understand what was implemented.

What to include in the code?

The code should contain a `README.md` file that describes how to run and reproduce your result. This includes the packages installed and their versions. We expected adequately commented code. To help with readability, please follow a style guide and use a linter, e.g., PEP8 is usually a good choice.

How to submit?

- You should have received an e-mail with the link to access Gradescope. If you haven't, let the TAs know.
- For your pdf file, use the naming convention `username_hw#.pdf`. For example, your TA with username `alice` would name their pdf file for HW2 as `alice_hw2.pdf`.
- Use the provided LaTeX template to typeset the assignment by editing the tex files under `student_response/`.
- After uploading your submission to Gradescope, mark each page to identify which question is answered on the page. (Gradescope will facilitate this.)
- For your code, use the naming convention `username_hw#.zip`. The code can be submitted via Gradescope under `assignment2_programming`.